

Übungsblatt 4: Race Conditions, Mutex und Priority Inversion

Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Zielsetzung

In dieser Laborübung machen Sie die Gefahren von ungeschützten Zugriffen auf geteilte Ressourcen (*Shared Resources*) sichtbar. Sie erzeugen zunächst eine *Race Condition* am seriellen Port (UART) und beheben diese anschließend mithilfe eines CMSIS-Mutexes. Danach provozieren Sie absichtlich eine *Priority Inversion* — das klassische Fehlerbild der Mars-Pathfinder-Mission — und beweisen, dass der Mutex das System durch *Priority Inheritance* (Prioritätsvererbung, PIP) retten kann.

Aufgabe 1: Das Chaos am UART (Race Condition)

Bauen Sie ein System, in dem zwei Tasks asynchron auf dieselbe Hardware-Ressource (den UART / `printf()`) zugreifen.

1. Legen Sie im CubeMX-Configurator zwei Tasks an:
 - `Task1_Normal` (Priorität: `osPriorityNormal`)
 - `Task2_High` (Priorität: `osPriorityHigh`)
2. Lassen Sie beide Tasks in ihren jeweiligen `for(;;)`-Schleifen einen langen Text (z. B. "`Task 1 sendet in Ruhe Daten...\n`") via `printf()` ausgeben. Fügen Sie anschließend ein `osDelay(100)` in beide Tasks ein.
3. Verwenden Sie **keinerlei Absicherung** (weder Mutex noch Semaphor).
4. Flashen Sie den Code und beobachten Sie die serielle Konsole Ihres PCs. Was fällt Ihnen an der Ausgabe auf? Erklären Sie die Ursache.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Aufgabe 2: Die Absicherung (Mutex)

Wir reparieren das Chaos aus Aufgabe 1.

1. Legen Sie im Code (z. B. vor den Task-Funktionen) einen Mutex an: `uartMutex = osMutexNew(NULL);`.
2. Umschließen Sie den `printf()`-Aufruf in **beiden** Tasks mit `osMutexAcquire()` (Timeout: `osWaitForever`) und `osMutexRelease()`.
3. **Achtung:** Das `osDelay()` muss **außerhalb** des Mutex-Blocks (der kritischen Sektion) stehen! Andernfalls halten Sie den Mutex während der Wartezeit und blockieren die andere Task unnötig lange.
4. Flashen Sie den Code und überprüfen Sie die Konsolenausgabe.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Aufgabe 3: Die Sabotage (Priority Inversion)

Wir bauen nun den fatalen Fehler des Mars Pathfinder nach. Pathfinder litt unter Priority Inversion, weil Prioritätsvererbung (PIP) damals deaktiviert war. Im FreeRTOS-/CMSIS-Labor erreichen wir denselben Effekt didaktisch sauber, indem wir den Mutex durch einen **binären Semaphor** ersetzen — Semaphoren besitzen weder Ownership noch PIP.

1. Ersetzen Sie in Ihrem Code den Aufruf `osMutexNew` durch einen binären Semaphor:

```
uartSem = osSemaphoreNew(1, 1, NULL); /* MaxCount=1, InitialCount=1 */
```

Passen Sie Handle-Typ und API an (`osSemaphoreId_t`, `osSemaphoreAcquire` / `osSemaphoreRelease`). Das Schutzmuster bleibt gleich: `Acquire` → `printf` → `Release` → `osDelay` (Delay weiterhin außerhalb!).

2. Fügen Sie im Configurator eine dritte Task hinzu: `Task3_Medium` (Priorität: `osPriorityAboveNormal` — also zwischen Normal und High).
3. Schreiben Sie in `Task3_Medium` eine reine CPU-Fresser-Schleife (wie in der letzten Laborübung):

```
volatile uint32_t x = 0;
while (1) { x++; }
```

Nutzen Sie **kein** Delay und fassen Sie den Semaphor **nicht** an!

4. Flashen Sie den Code und beobachten Sie die Konsole.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Aufgabe 4: Die Heilung (Priority Inheritance)

Wir retten unser System durch eine einzige API-Änderung — analog zum NASA-Patch der Pathfinder-Mission.

1. Ersetzen Sie den fehlerhaften Semaphor im Code wieder durch den korrekten CMSIS-Mutex: `uartMutex = osMutexNew(NULL);`. Ändern Sie die Task-Logik der drei Tasks sonst nicht (Acquire → `printf` → Release → `osDelay` außerhalb; Medium-Task bleibt CPU-Fresser).
2. Flashen Sie den Code neu.
3. Erklären Sie, warum das System nun wieder funktioniert, obwohl `Task3_Medium` weiterhin 100 % CPU-Last fordert.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)